

Locally-Sensitive Hashing

IIC2440 — Procesamiento de Datos Masivos

Juan Reutter

Agenda

1. Motivación
2. LSH: idea general
3. LSH para distancia angular
4. Amplificación AND/OR
5. Bases de datos de vectores: LSH Forest

Clases anteriores:

- Sampling: trabajar con un subconjunto de los datos.
- Hashing (Bloom, Count-Min): "¿está este elemento? ¿cuántas veces apareció?"

Hoy: hashing que **preserva distancia**.

Cambiamos la pregunta: en vez de *¿está este elemento?*, queremos *¿qué elementos son parecidos a este?*

Esto es la base de las bases de datos de vectores, búsqueda por similitud, y los sistemas de retrieval detrás de RAG.

El problema: pares similares

El problema

Dada una colección de n elementos y una noción de distancia d , encontrar los pares (x, y) con $d(x, y)$ pequeña.

Aplicaciones:

- Deduplicación de documentos / detección de plagio
- Productos similares en un e-commerce
- Imágenes parecidas
- Vecinos cercanos en un embedding (RAG, recomendación)

¿Por qué es difícil?

La solución trivial: calcular $d(x, y)$ para **cada par** (x, y) .

Eso es $\binom{n}{2} \approx n^2/2$ cálculos.

Con $n = 10^6$ elementos: 5×10^{11} pares. A $1 \mu\text{s}$ por distancia: **~6 días** de cómputo.

Con $n = 10^9$: imposible en la vida de un humano.

Necesitamos una técnica que descarte **la mayoría de los pares** sin calcular su distancia.

La estrategia que vamos a desarrollar

Ya usamos hashing para responder *¿está este elemento?* (Bloom) y *¿cuántas veces apareció?* (Count-Min).

Hoy vamos a inventar hashes con una propiedad nueva:

Si dos elementos son parecidos, queremos que su hash sea el mismo con alta probabilidad.

Si lo logramos, el resto es fácil: agrupamos por hash y solo comparamos pares dentro de cada grupo. Toda la gracia es construir esa función de hash y entender qué tan bien funciona.

LSH: idea general

Distancias

Una función d sobre pares de elementos es una **distancia** si:

- $d(x, x) = 0$
- $d(x, y) > 0$ para $x \neq y$
- $d(x, y) = d(y, x)$ (simetría)
- $d(x, z) \leq d(x, y) + d(y, z)$ (desigualdad triangular)

Un conjunto equipado con una distancia es un **espacio métrico**.

Ejemplos: euclidiana en $\mathbb{R}^n$, angular en $\mathbb{R}^n$, Jaccard sobre conjuntos, Hamming sobre strings.

Cuatro distancias, un ejemplo de cada una

- **Jaccard.** $A = \{\text{pan, leche, palta}\}, B = \{\text{pan, leche, queso, huevo}\}.$
 $|A \cap B| = 2, |A \cup B| = 5, d_J = 1 - 2/5 = 0,6.$
- **Hamming.** "casa" vs "cosa" $\rightarrow$ difieren en 1 de 4 posiciones, $d_H = 1.$
- **Angular.** $\bar{u} = (1, 0), \bar{v} = (1, 1) \rightarrow \cos \theta = 1/\sqrt{2}, \theta = 45^\circ.$
- **Euclidiana.** Los mismos vectores $\rightarrow \|\bar{u} - \bar{v}\| = 1.$

Notemos que "cerca" significa cosas muy distintas en cada espacio.

LSH tendrá una familia de hash distinta para cada distancia.

La idea central de LSH

Diseñar una función de hash f tal que **elementos cercanos tienen más probabilidad de colisionar** que elementos lejanos.

Si lo logramos, la receta es:

1. Calcular $f(e)$ para cada e en la colección.
2. Para cada par (x, y) con $f(x) = f(y)$, calcular $d(x, y)$.
3. Reportar los que tengan distancia baja.

Ganancia: saltamos el cálculo de distancia para la mayoría de los pares que no colisionan (y asumimos que esos estaban lejos).

Imaginemos 6 documentos $d_1, \dots, d_6$ y una función de hash inventada f que devuelve un color. Supongamos que obtenemos:

documento	f
d_1, d_3, d_4	rojo
d_2, d_6	azul
d_5	verde

En vez de los $\binom{6}{2} = 15$ pares posibles, solo calculamos distancia para pares **dentro del mismo color**: $(d_1, d_3), (d_1, d_4), (d_3, d_4), (d_2, d_6)$.

Pasamos de 15 a 4 cálculos. La promesa de LSH es que esos 4 contienen casi todos los pares realmente cercanos.

Familias sensitivas

Definición. Una familia $\mathcal{F}$ de funciones $E \rightarrow R$ es (d_1, d_2, p_1, p_2) -**sensitiva** si para todo $x, y \in E$ y $f \in \mathcal{F}$ tomada al azar:

- Si $d(x, y) \leq d_1$: $P(f(x) = f(y)) \geq p_1$
- Si $d(x, y) \geq d_2$: $P(f(x) = f(y)) \leq p_2$

Queremos $d_1 < d_2$ y $p_1 > p_2$ — o sea, **pares cercanos colisionan más**.

El **gap** $(p_1 - p_2)$ mide qué tan buena es la familia.

Ejemplo: distancia de Jaccard, MinHash

Para conjuntos A, B :

$$\text{Jaccard}(A, B) = \frac{|A \cap B|}{|A \cup B|} \quad d_J(A, B) = 1 - \text{Jaccard}(A, B)$$

MinHash. Sea h una función de hash con valores en $\mathbb{R}$. Definimos $f_h(A) = \min_{a \in A} h(a)$.

Resultado clave:

$$P(f_h(A) = f_h(B)) = \text{Jaccard}(A, B)$$

→ familia $(d_1, d_2, 1 - d_1, 1 - d_2)$ -sensitiva. Usada para deduplicación a gran escala (Google, Common Crawl).

Por qué MinHash funciona (intuición)

Pensemos en $A \cup B$ y miremos qué elemento de ese conjunto recibe el **mínimo valor** bajo h .

Como h es aleatoria, ese mínimo cae en cualquier elemento de $A \cup B$ con igual probabilidad.

Ahora: $f_h(A) = f_h(B)$ ocurre exactamente cuando ese elemento minimal está en $A \cap B$ — solo así es el mínimo tanto de A como de B . Por lo tanto:

$$P(f_h(A) = f_h(B)) = \frac{|A \cap B|}{|A \cup B|} = \text{Jaccard}(A, B)$$

Patrón común: "definir f , mostrar que $P(\text{colisión})$ es función de la distancia".

LSH para distancia angular

Distancia angular

Para vectores $\bar{u}, \bar{v} \in \mathbb{R}^n$:

Similaridad de coseno:

$$\text{sc}(\bar{u}, \bar{v}) = \frac{\bar{u} \cdot \bar{v}}{\|\bar{u}\| \|\bar{v}\|} \in [-1, 1]$$

Distancia angular (en grados):

$$d_a(\bar{u}, \bar{v}) = \arccos(\text{sc}(\bar{u}, \bar{v})) \in [0^\circ, 180^\circ]$$

- $d_a = 0^\circ$: misma dirección (hasta escala).
- $d_a = 180^\circ$: direcciones opuestas.

Usado en: embeddings de texto (Word2Vec, BERT, OpenAI embeddings), recomendadores, imágenes.

Esquema LSH angular

Sea $\bar{w}$ un vector aleatorio. Definimos:

$$f_{\bar{w}}(\bar{u}) = \text{signo}(\bar{w} \cdot \bar{u}) \in \{+1, -1\}$$

Equivalentemente: $f_{\bar{w}}(\bar{u}) = f_{\bar{w}}(\bar{v})$ si y solo si $\bar{u}$ y $\bar{v}$ están **del mismo lado** del hiperplano normal a $\bar{w}$.

La familia es:

$$\mathcal{F} = \{f_{\bar{w}} : \bar{w} \in \mathbb{R}^n\}$$

Geometría (en pizarra)

$\bar{u}$ y $\bar{v}$ definen un plano, con ángulo θ entre ellos.

Cada hiperplano aleatorio "corta" el plano en dos mitades. Nuestro hash $f_{\bar{w}}$ dice de qué mitad está cada vector.

La pregunta clave es geométrica:

¿Con qué probabilidad un hiperplano aleatorio pasa entre $\bar{u}$ y $\bar{v}$?

La respuesta depende solo del ángulo θ . Vamos a calcularla.

Intuición geométrica: el cálculo

Proyectemos $\bar{w}$ al plano definido por $\bar{u}, \bar{v}$. Su dirección φ es **uniforme en** $[0^\circ, 360^\circ)$

- $\bar{w} \cdot \bar{u} > 0 \iff \varphi$ está a menos de 90° de $\bar{u}$.
- Los signos difieren $\iff \varphi$ cae en uno de dos arcos de ancho θ (uno entre $\bar{u}$ y $\bar{v}$, el otro entre $-\bar{u}$ y $-\bar{v}$).

$$P(f_{\bar{w}}(\bar{u}) \neq f_{\bar{w}}(\bar{v})) = \frac{2\theta}{360^\circ} = \frac{\theta}{180^\circ}$$

Por lo tanto:

$$P(f_{\bar{w}}(\bar{u}) = f_{\bar{w}}(\bar{v})) = \frac{180 - \theta}{180}$$

Mismo patrón que MinHash: definimos f y obtenemos $P(\text{colisión})$ como función limpia de la distancia.

La familia es sensitiva

Para $0 \leq d_1 < d_2 \leq 180$, la familia $\mathcal{F}$ es:

$$\left(d_1, d_2, \frac{180 - d_1}{180}, \frac{180 - d_2}{180} \right)\text{-sensitiva}$$

Verificación. Si $d_a(\bar{u}, \bar{v}) \leq d_1$: probabilidad de misma firma es $(180 - d_a)/180 \geq (180 - d_1)/180$. ✓

Si $d_a(\bar{u}, \bar{v}) \geq d_2$: probabilidad de misma firma es $(180 - d_a)/180 \leq (180 - d_2)/180$.
✓

Pero no basta

Supongamos que queremos pares con distancia $\leq 20^\circ$. Con una sola función:

$$p_1 = \frac{160}{180} \approx 0,889 \quad \implies \quad \text{falsos negativos} \approx 11,1\%$$

Es demasiado. Vamos a dejar fuera 1 de cada 9 pares cercanos.

Y por otro lado, dos vectores a distancia 90° (ortogonales, "distintos") colisionan con probabilidad 0,5 — calcularemos demasiadas distancias.

Necesitamos **amplificar**: subir p_1 , bajar p_2 .

Tipos de error

Si usamos la receta para reportar pares con $d(x, y) \leq d_1$:

- **No hay falsos positivos:** antes de reportar, calculamos la distancia real.
- **Puede haber falsos negativos:** pares cercanos con $f(x) \neq f(y)$ (probabilidad $\leq 1 - p_1$).

Siguiendo el ejemplo anterior: el 11,1% de falsos negativos es exactamente $1 - p_1$, y la colisión espuria a $\theta = 90^\circ$ con probabilidad 0,5 es p_2 .

Reducir falsos negativos pide p_1 alto; pero eso suele venir con p_2 también alto (muchas distancias calculadas en vano).

El trabajo que sigue: **amplificar el gap** entre p_1 y p_2 .

Amplificación AND/OR

Construcción OR

Tomamos b funciones $f_1, \dots, f_b \in \mathcal{F}$ y definimos:

$$f^{\text{OR}}(x, y) = 1 \iff \exists i : f_i(x) = f_i(y)$$

"Colisionan si colisionan en **alguna**."

Efecto en las probabilidades: si $\mathcal{F}$ era (d_1, d_2, p_1, p_2) -sensitiva, entonces $\mathcal{F}^{\text{OR}}$ es:

$$(d_1, d_2, 1 - (1 - p_1)^b, 1 - (1 - p_2)^b)\text{-sensitiva}$$

Sube ambas probabilidades p_1 y p_2 (lo ideal sería solo p_1).

Construcción AND

Tomamos r funciones $f_1, \dots, f_r \in \mathcal{F}$ y definimos:

$$f^{\text{AND}}(x, y) = 1 \iff \forall i : f_i(x) = f_i(y)$$

"Colisionan si colisionan en **todas**."

Efecto en las probabilidades: $\mathcal{F}^{\text{AND}}$ es:

(d_1, d_2, p_1^r, p_2^r) -sensitiva

Baja ambas probabilidades (lo ideal sería solo p_2).

Ninguna construcción sola sirve. La magia está en **combinarlas**.

AND-OR: esquema de bandas

Organizamos $b \cdot r$ funciones en b **bandas** de r funciones cada una.

- Dentro de cada banda: AND (deben coincidir **todas** las r).
- Entre bandas: OR (basta con que coincida **alguna** banda).

Equivalente: $\mathcal{F}^{\text{AND}}$ primero, luego $\mathcal{F}^{\text{OR}}$.

Probabilidad final de que x, y sean candidatos:

$$P_{b,r}(p) = 1 - (1 - p^r)^b$$

donde p es la probabilidad original de colisión según la distancia.

Caso angular

Para vectores a distancia angular θ , $p = (180 - \theta)/180$. La probabilidad final:

$$\text{LSH}_{b,r}(\theta) = 1 - \left(1 - \left(\frac{180 - \theta}{180}\right)^r\right)^b$$

Con b, r bien elegidos obtenemos una **curva S**: alta probabilidad para θ pequeño, baja para θ grande, transición abrupta en el medio.

Cada vector recibe $b \cdot r$ hashes; para consultar, buscamos matches banda por banda. El costo es lineal en la colección, no cuadrático.

La curva S

$\text{LSH}_{b,r}(\theta) = 1 - \left(1 - \left(\frac{180-\theta}{180}\right)^r\right)^b$ tiene forma de **S** en θ .

Para $(b = 10, r = 10)$:

θ	$p = \frac{180-\theta}{180}$	$P(\text{candidato})$
10°	0,944	0,9998
20°	0,889	0,975
30°	0,833	0,825
45°	0,750	0,441
60°	0,667	0,160
90°	0,500	0,010

Transición nítida cerca de $\theta \approx 45^\circ$. Casi todos los pares bajo 30° se reportan; casi nada

Comparando configuraciones

Mismo umbral, distintas elecciones de (b, r) . Probabilidad de ser candidato:

(b, r)	$\theta = 30^\circ$	$\theta = 45^\circ$	$\theta = 60^\circ$
$(10, 5)$	0,994	0,910	0,634
$(10, 10)$	0,825	0,441	0,160
$(50, 20)$	0,541	0,015	$\sim 10^{-5}$

- **Subir r** mueve la transición a la izquierda (más estricto, descarta más).
- **Subir b** la mueve a la derecha (más permisivo, deja pasar más).

El truco es elegir b y r **juntos** para ubicar la transición donde nos importa.

Elegir el punto de transición

Donde la curva cruza $P = 1/2$:

$$1 - (1 - p^r)^b = \frac{1}{2} \iff p^r = 1 - 2^{-1/b}$$

Usando $1 - 2^{-1/b} \approx (\ln 2)/b$ para b moderado/grande:

$$p^* \approx \left(\frac{\ln 2}{b} \right)^{1/r}$$

Receta. Fijar el umbral de distancia θ^* a partir del cual dejan de importar los pares; despejar $p^* = (180 - \theta^*)/180$ y elegir b, r que ubiquen la transición ahí.

Trade-offs de b y r

- $b \cdot r$ grande $\rightarrow$ firma más pesada y más trabajo por consulta, pero curva S más nítida.
- r grande (AND) $\rightarrow$ baja p_2 (menos falsos positivos) pero también p_1 (más falsos negativos).
- b grande (OR) $\rightarrow$ sube p_1 pero también p_2 .

Están acoplados: subir r para ganar nitidez requiere subir b para compensar.

Bases de datos de vectores

De LSH a estructura de búsqueda

LSH nos da hashes que **preservan distancia**. ¿Cómo usarlos para **consultas online**?

Dado un vector e , devolver los K elementos más cercanos en la colección.

Idea: indexar la colección por su firma LSH, y para consultar computar la firma de e y mirar "cerca de" su firma.

La estructura clásica: **LSH Forest**.

LSH Tree

Tomamos ℓ funciones binarias $f_1, \dots, f_\ell$ (para angular: signo del producto con un vector aleatorio). A cada elemento e le asociamos:

$$\text{sig}(e) = (f_1(e), f_2(e), \dots, f_\ell(e)) \in \{0, 1\}^\ell$$

Construimos un **árbol binario de búsqueda** sobre estos bitvectors:

- Nodo raíz tiene hijos n_0 y n_1 (según f_1).
- n_0 tiene hijos n_{00}, n_{01} (según f_2), y así.
- Las hojas son los 2^ℓ bitvectors posibles; cada hoja apunta a los elementos con esa firma.

Búsqueda en un LSH Tree

Para buscar los K vecinos más cercanos de e :

1. Calcular $\text{sig}(e)$.
2. Ir a la hoja correspondiente: devolver todos los elementos ahí.
3. Si aún no hay K : subir un nivel (hermanos comparten $\ell - 1$ bits iniciales) y agregar esos elementos.
4. Repetir subiendo hasta reunir $\geq K$ candidatos.
5. (Opcional) Ordenar los candidatos por distancia real.

Intuición: subir un nivel equivale a relajar el criterio de colisión de AND de ℓ a AND de $\ell - 1$.

Búsqueda en acción ($K = 3$)

Árbol binario de profundidad 3 con elementos $e_1, \dots, e_8$ en las hojas. Consultamos e con $\text{sig}(e) = 011$.

Paso	Nodo visitado	Candidatos acumulados
1	hoja 011	$\{e_5\}$
2	subimos a 01*	$\{e_5, e_4\}$
3	subimos a 0**	$\{e_5, e_4, e_1, e_2\}$
4	distancia real con los 4	devolvemos los 3 más cercanos

Cada nivel que subimos **relaja un AND**: pasamos de coincidir en 3 bits, a 2, a 1. Es exactamente el OR del esquema de bandas, expresado como navegación de árbol.

LSH Forest: optimizaciones

Problema 1. El árbol tiene 2^ℓ hojas potenciales — prohibitivo para ℓ grande.

→ Construir el árbol **top-down solo desde los elementos que tenemos** (el subconjunto $S \subseteq E$). No hay hojas vacías.

Problema 2. Hojas con muy pocos elementos son inútiles.

→ Parámetro H : si una hoja tiene menos de H elementos, se **fusiona con su hermana** y los elementos suben al padre.

Problema 3. Un solo árbol tiene varianza alta.

→ Parámetro F : mantener F **árboles en paralelo** (con funciones independientes). Al consultar, buscar en todos y tomar la unión.

El árbol como AND-OR

Un LSH Forest con F árboles, cada uno de profundidad ℓ , es exactamente el esquema AND-OR:

- **AND dentro de cada árbol:** firma de ℓ bits = coincidencia en ℓ funciones.
- **OR entre árboles:** basta con coincidir en algún árbol.

$$P(\text{candidato}) = 1 - (1 - p^\ell)^F$$

Los parámetros H (tamaño mínimo de hoja) y la posibilidad de subir niveles al consultar son refinamientos prácticos.

Panorama industrial

LSH Forest es **una** técnica. La industria hoy usa varias:

Técnica	Idea	Librerías
LSH / LSH Forest	Hashing que preserva distancia	<code>datasketch</code> , <code>annoy</code>
HNSW	Grafos navegables de vecindades	<code>hnswlib</code> , <code>pgvector</code> , Milvus, Qdrant
IVF	Particionar el espacio con clustering	FAISS (Meta), ScaNN (Google)
Product Quantization	Comprimir vectores preservando distancia	FAISS

Sistemas de vector DB: Chroma, Weaviate, Qdrant, Pinecone, `pgvector`, Milvus.

Todos comparten la filosofía: **aproximar vecinos cercanos en sublineal** con alguna garantía probabilística.

Conexión con RAG

El motor detrás de Retrieval-Augmented Generation:

1. **Embedding**: cada documento $\rightarrow$ vector (ej: `multilingual-e5-small`, OpenAI `text-embedding-3-small`).
2. **Index**: guardar vectores en una base de datos de vectores.
3. **Query**: embedding de la pregunta $\rightarrow$ buscar los K más cercanos $\rightarrow$ pasar como contexto al LLM.

Sin LSH (o algún equivalente), el paso 3 sería $O(n)$ por consulta — imposible para colecciones grandes.

Resumen de las últimas 3 clases

Clase 05 — Sampling: trabajar con un subconjunto representativo.

Clase 06 — Hashing: mapear a un espacio más pequeño, preservar "membership" y "conteo".

Clase 07 — LSH: hashing que preserva **distancia**. Colisiones dejan de ser accidentes: las usamos para encontrar lo parecido.

Las tres técnicas comparten la filosofía: **cambiar exactitud por velocidad con probabilidad controlada de error.**

Todas son la base de sistemas reales que procesan terabytes y billones de eventos por día.